

Final Engineering Project: The Local RAG & Deployment Challenge

Objective: In this project, you will step into the role of a Backend AI Engineer. The NLP course has introduced you to the methods of processing unstructured data and context-aware generation. You will demonstrate your mastery of these themes by building an end-to-end, containerized Retrieval-Augmented Generation (RAG) system via a FastAPI backend to solve a specific domain problem (e.g., matching candidates to job requirements based on raw CVs, querying corporate legal manuals).

Part 1: Rules of Engagement (The Constraints)

To ensure you are building a genuine engineering pipeline rather than following a basic LangChain tutorial, your project must strictly adhere to the following rules:

- **The "No Toy Data" Rule:** You may not use pre-cleaned text files or generic Wikipedia CSVs. Your system must ingest messy, unstructured documents (e.g., raw PDFs, Word documents, scanned CVs, or HTML pages).
- **The "Dirty Hands" Rule:** You must actively process the data. This means you must write custom parsing logic, handle metadata, and mathematically justify your **Chunking Strategy** (e.g., why 500 tokens with a 50-token overlap? Did you chunk by semantic sentences or just hard limits?).
- **The Containerization Mandate:** Your final system cannot just be a Jupyter Notebook. The RAG logic and the Vector Database must be wrapped in a **FastAPI** backend and containerized using **Docker**. Your TA must be able to run your entire system using a simple `docker-compose up` command.

Part 2: The Engineering Pipeline

Deliverables:

- 1- GitHub Repository (The Code).
- 2- Technical Report (The Documentation) covering core phases and how you encountered them.

Phase 1: Data Processing & Vectorization

Formulate a research question/querying problem. Parse your unstructured data, apply your chunking strategy, vectorize the text using an appropriate embedding model, and store it in a Vector Database (e.g., ChromaDB, FAISS, Qdrant).

Phase 2: System Architecture (FastAPI & Docker)

Build a robust API interface for your RAG logic using FastAPI. You are expected to apply clean software engineering principles (such as the MVC pattern) to separate your routing, your RAG logic, and your data models. Finally, package the application using a Dockerfile and/or `docker-compose.yml` to ensure it runs consistently across environments.

Phase 3: The RAG Logic & Retrieval

Implement the querying pipeline. Your API must accept a user query, embed it, perform a vector search to retrieve the top-k most relevant chunks, inject those chunks into an LLM context window, and return the generated answer via the API endpoint.

Phase 4: Evaluation & Error Analysis

RAG systems are prone to hallucination and poor retrieval. You must evaluate your system's **Retrieval Accuracy**. Present at least 3 edge-case queries where the system failed (e.g., it retrieved the wrong chunk, or the LLM hallucinated despite the context) and explain *why* the architecture missed the mark.

Phase 5: The Technical Report (Documentation)

Submit a standalone 1-2 page Markdown or PDF document. This is your "Tech Spec." It must include an Executive Summary, a system architecture diagram, your API documentation (endpoints and expected JSON payloads), a justification of your embedding model and chunking strategy, and clear Docker deployment instructions.

★ Bonus Phases (Up to +15%)

Push your system to a true production standard by implementing one or both of these advanced features:

- **Bonus 1: The LLM Factory Pattern (+5%):** Instead of hardcoding a single model, implement a Factory Design Pattern. Create a unified interface that allows the system to switch between different local models (e.g., Ollama/Mistral) or remote APIs (e.g., Gemini/OpenAI) with a single configuration change.
- **Bonus 2: Arabic Language Support (+10%):** Account for the specific challenges of Arabic NLP (e.g., Arabic CVs). Discuss how your system handles Arabic script normalization and RTL text extraction from PDFs. *Requirement: You must include at least one Arabic test case/query and discuss its results in your final report.*

Grading Rubric (100 Points Total)

Category	Weight	What We Are Looking For
Data Processing & Chunking	20%	Did you handle raw, unstructured data? Is your chunking strategy mathematically and semantically justified?
Backend & Architecture	25%	Is the FastAPI code clean and well-structured (MVC)? Are the API endpoints functional and returning correct payloads?
Containerization (Docker)	25%	Does the Dockerfile / docker-compose.yml work flawlessly? Are dependencies managed correctly?
Retrieval & RAG Logic	20%	Is the context injection accurate? Did you provide a robust error analysis of failed retrievals and hallucinations?
Technical Documentation	10%	Does the report cover the architecture, data choices, and Docker instructions? Can another engineer run your code based on the docs?
[BONUS] Advanced Features	+15%	Did you successfully implement the LLM Factory Pattern and/or Multilingual Arabic parsing?

Notes:

- Any extra steps, work, etc. will be considered as extra bonus (relative to your effort and TA evaluation).
- You can use Google Colab or Kaggle notebook not necessarily your own hardware.
- Please register your team and project of choice through this form:
<https://forms.gle/DCBUhw7QWAczARh49>